

PART 4: SEQUENTIAL LOGIC

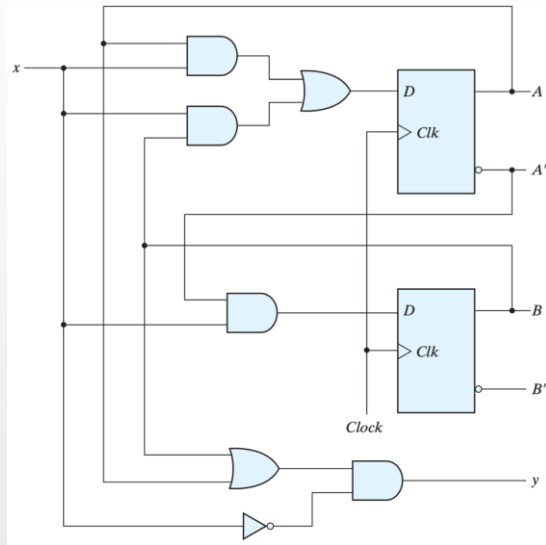
FINITE STATE MACHINES

SAAD BAIG – DIGITAL LOGIC DESIGN



The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>

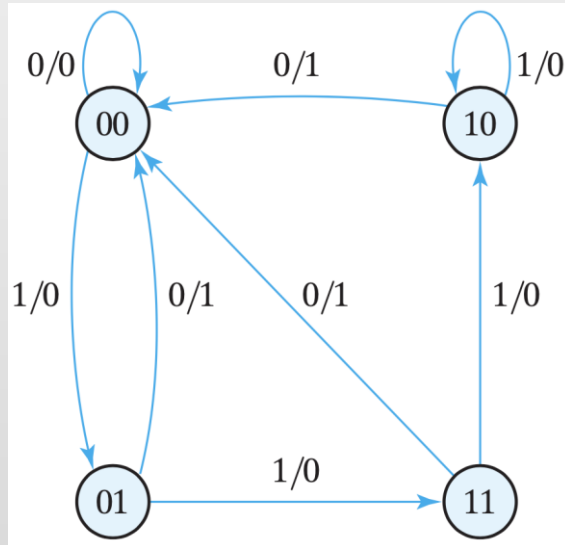


$$A(t + 1) = Ax + Bx$$

$$B(t + 1) = A'x$$

$$y(t) = (A + B)x'$$

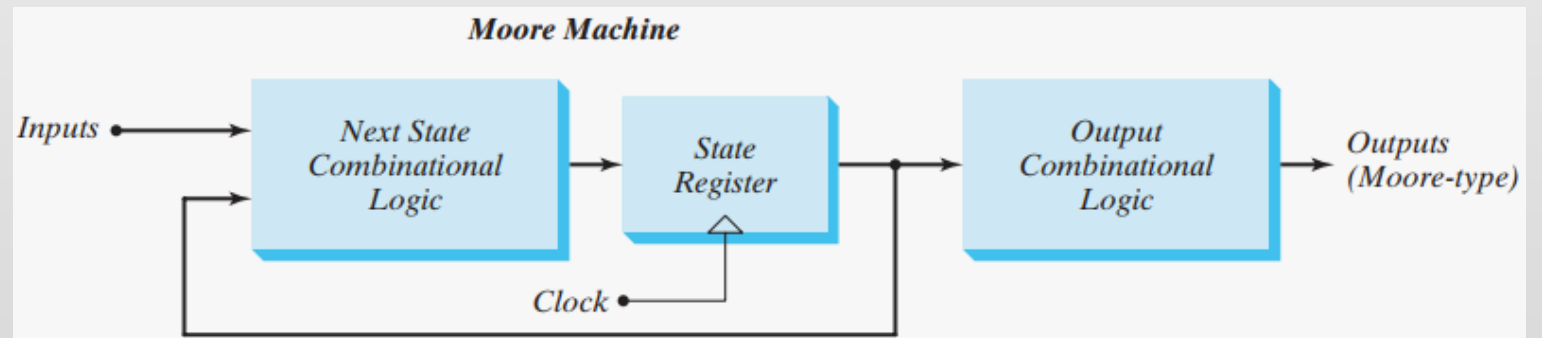
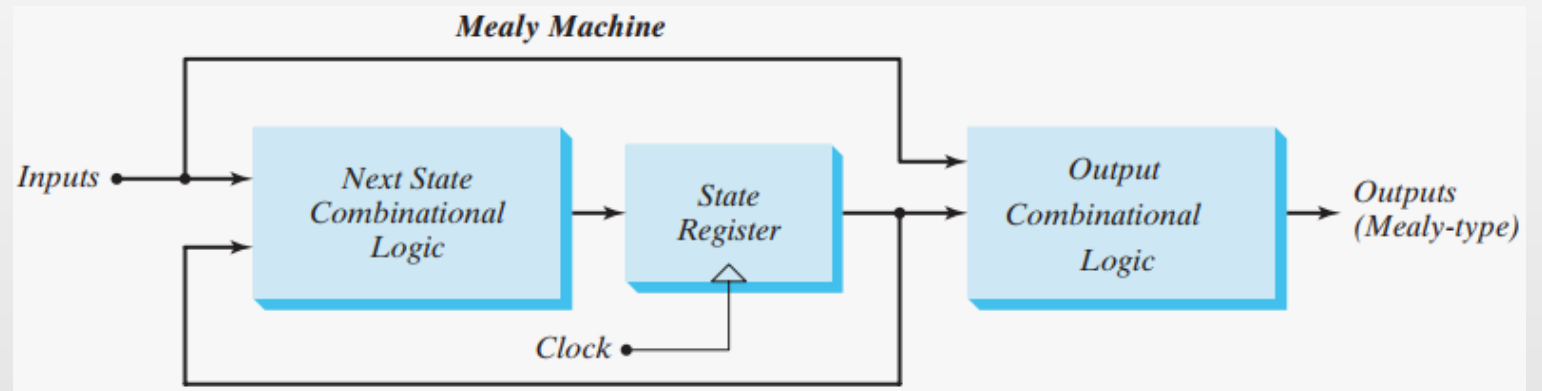
Present State		Input x	Next State		Output y
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0



Present State		Next State				Output	
		$x = 0$		$x = 1$		$x = 0$	$x = 1$
A	B	A	B	A	B	y	y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

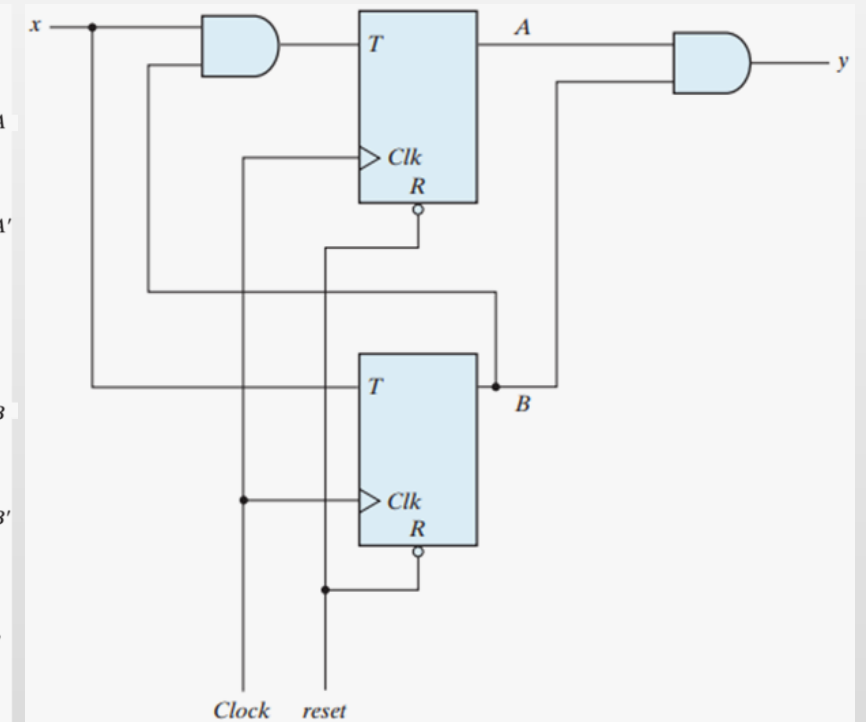
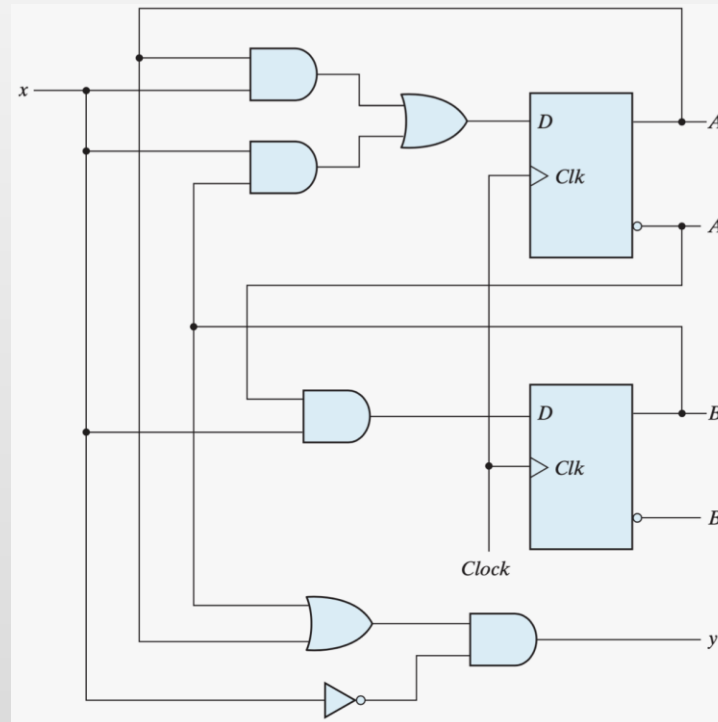
FINITE STATE MACHINES

- Mealy Machine:
 - Output is function of both input and present state. May have momentary false values because of delay between inputs change and flip-flop outputs change.
- Moore Machine:
 - Output is function of present state only. Outputs are synchronized with clock.



FINITE STATE MACHINES

- Mealy Machine:
 - Output is function of both input and present state. May have momentary false values because of delay between inputs change and flip-flop outputs change.
- Moore Machine:
 - Output is function of present state only. Outputs are synchronized with clock.





2-BIT BINARY COUNTER USING FSM

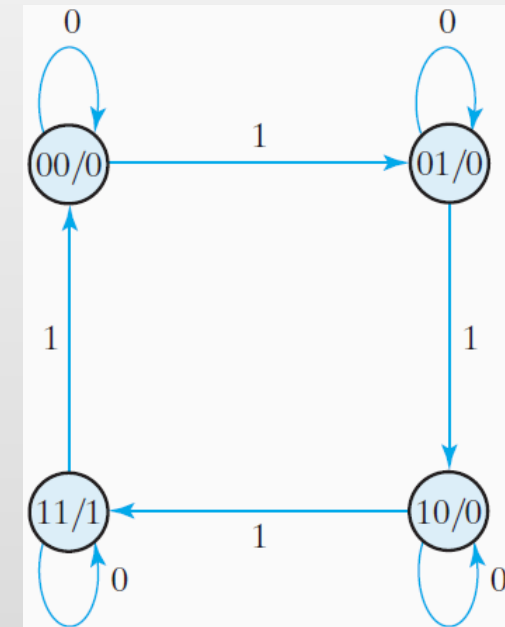
- Let's create a 2 bit binary counter with FSM.
 - It would count from 0 to 3 with a high input, otherwise the count will remain the same. At the highest number, light up an LED.
- Would this be a Mealy Machine or a Moore Machine?
- First step identifying states
- Second step, identifying inputs
- Third step, identifying outputs

2-BIT BINARY COUNTER USING FSM

State Table:

Present State		Input	Next State		Output
A	B		A	B	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

State Diagram:



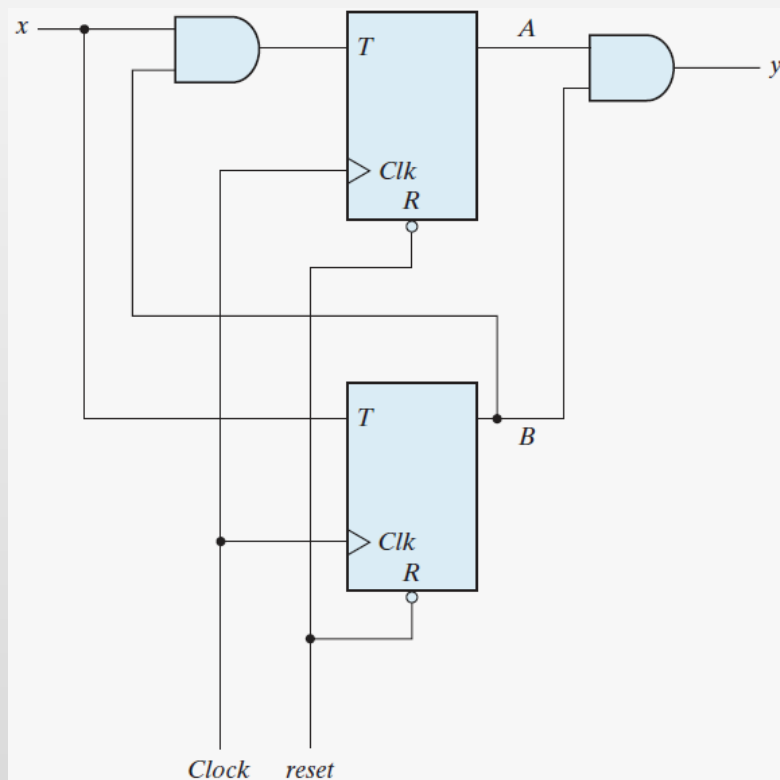


2-BIT BINARY COUNTER USING FSM

- Boolean Expressions?

Present State		Input	Next State		Output
<i>A</i>	<i>B</i>		<i>A</i>	<i>B</i>	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1

2-BIT BINARY COUNTER USING FSM



Present State		Input <i>x</i>	Next State		Output <i>y</i>
<i>A</i>	<i>B</i>		<i>A</i>	<i>B</i>	
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	0	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	1



FINITE STATE MACHINE DESIGN PROCEDURE

1. Understand the problem.
 - Try input sequences to be sure you understand the conditions under which the FSM transitions between states and outputs are asserted.
2. Obtain an abstract representation of the FSM (e.g. state diagram).
3. Perform state minimization
4. Perform state assignment.
 - A good choice of how to encode the state often leads to simpler implementation.
5. Implement the finite state machine



STATE ASSIGNMENT

- In order to design a sequential circuit with physical components, it is necessary to assign unique coded binary values to the states.
- For a circuit with m states, the codes must contain n bits, where $2^n \geq m$.
- Unused states are treated as don't care conditions in design.

Three Possible Binary State Assignments

State	Assignment 1, Binary	Assignment 2, Gray Code	Assignment 3, One-Hot
<i>a</i>	000	000	00001
<i>b</i>	001	001	00010
<i>c</i>	010	011	00100
<i>d</i>	011	010	01000
<i>e</i>	100	110	10000



STATE ASSIGNMENT EXAMPLE

Reduced State Table

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
a	a	b	0	0
b	c	d	0	0
c	a	d	0	0
d	e	d	0	1
e	a	d	0	1

Reduced State Table with Binary Assignment 1

Present State	Next State		Output	
	$x = 0$	$x = 1$	$x = 0$	$x = 1$
000	000	001	0	0
001	010	011	0	0
010	000	011	0	0
011	100	011	0	1
100	000	011	0	1

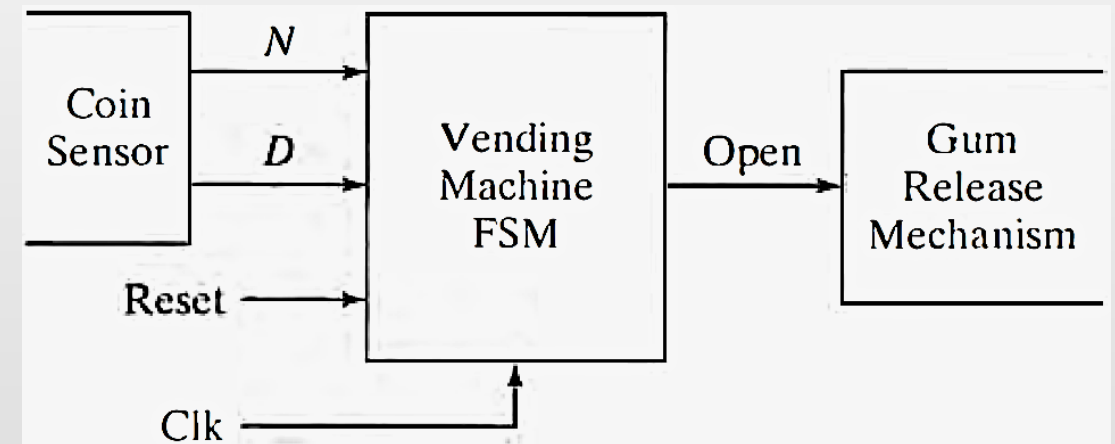


A SIMPLE VENDING MACHINE

- Statement:
 - The vending machine delivers bubble gum after it has received 15 cents in coins.
 - A single coin slot accepts 5 cents (nickels) and 10 cents (dimes) at a time.
 - A mechanical sensor indicates whether a nickel or a dime has been inserted into the coin slot.
 - The controller output causes a single item to be released down a chute to the customer.
- Assumptions:
 - No change policy (a person who pays with two dimes will lose the extra 5 cents).
 - We will expect our machine to be reset before each new use. This may be accomplished through a spate mechanism the customer uses to select the item to be purchased.

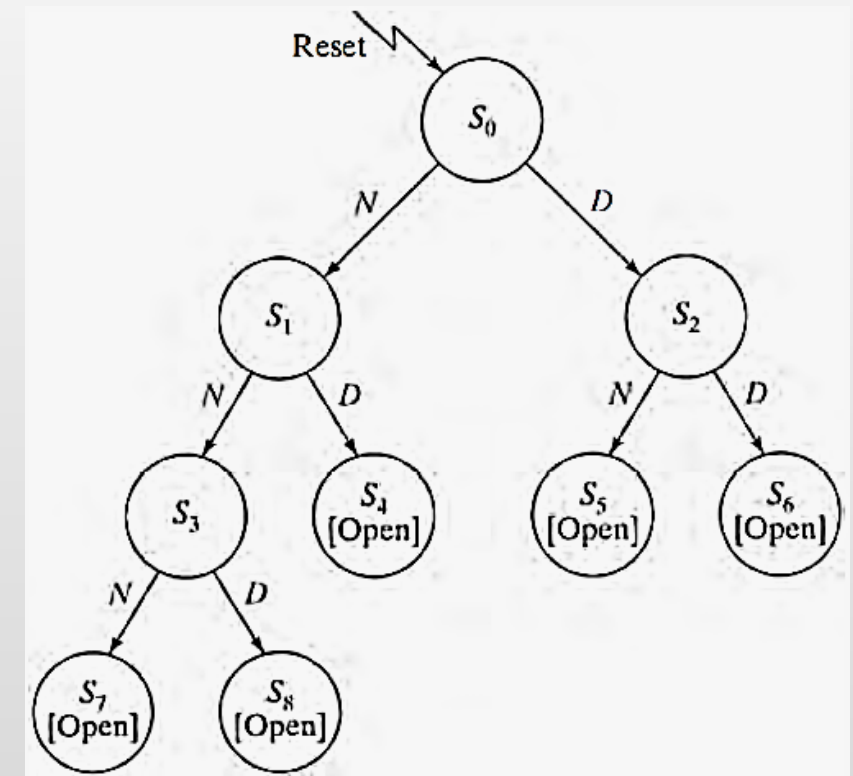
UNDERSTAND THE PROBLEM

- Start by drawing a block diagram to understand inputs and outputs.
 - N = Nickels, D = Dimes
- What happens if someone inserts a penny into the coin slot?
- Let's assume options other than nickels and dimes are ignored, leaving N and D unasserted.
- In design, it is crucial to have assumptions aired early, and keep track of them as design progresses.



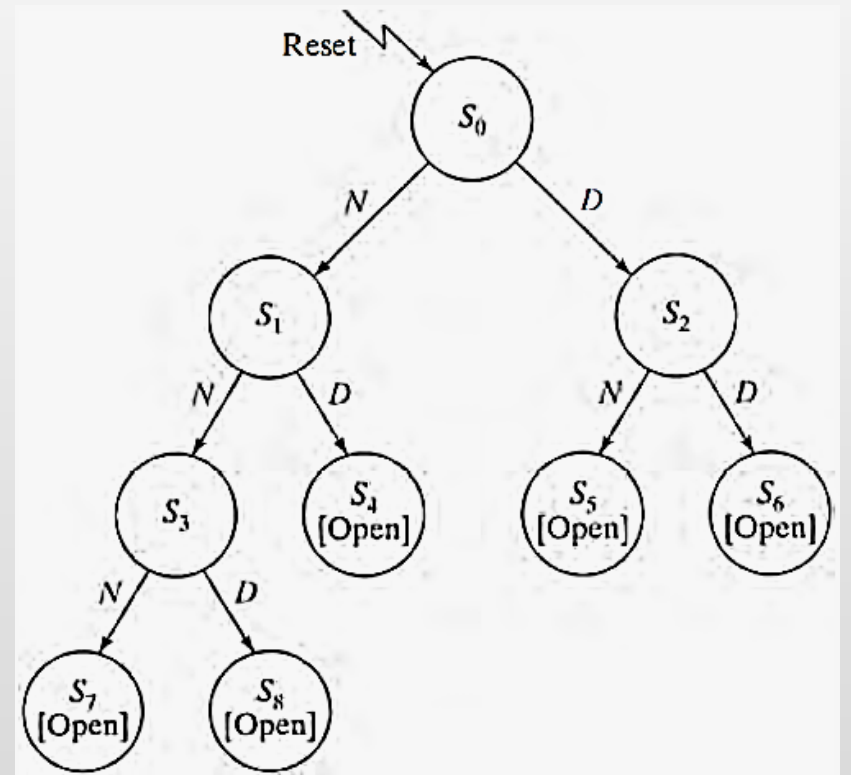
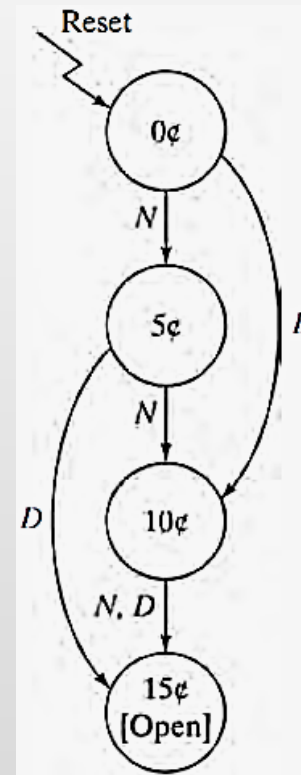
ABSTRACT REPRESENTATIONS

- Map the specification to a more suitable representation.
 - 3 nickels: N, N, N
 - 2 nickels 1 dime: N, N, D
 - 1 nickel 1 dime: N, D
 - 1 dime 1 nickel, D, N
 - 2 dimes: D, D
- Only transitions that cause a state change are included.
 - We assume that state remains same for no coin.



STATE MINIMIZATION

- S_4, S_5, S_6, S_7 and S_8 have identical behavior. They can be combined into a single state.
- We can also think of each state as representing the amount of money received so far.
 - It doesn't matter whether it took 1 dime or 2 nickels to reach a state with 10 cents.



STATE ENCODING

- The way states are encoded can have a major impact on the amount of combinational logic we need.

Present State	Inputs <i>D N</i>		Next State	Output Open
0¢	0	0	0¢	0
	0	1	5¢	0
	1	0	10¢	0
	1	1	X	X
5¢	0	0	5¢	0
	0	1	10¢	0
	1	0	15¢	0
	1	1	X	X
10¢	0	0	10¢	0
	0	1	15¢	0
	1	0	15¢	0
	1	1	X	X
15¢	X	X	15¢	1

Present State <i>Q₁ Q₀</i>		Inputs <i>D N</i>		Next State <i>D₁ D₀</i>		Output Open
0	0	0	0	0	0	0
		0	1	0	1	0
		1	0	1	0	0
		1	1	X	X	X
0	1	0	0	0	1	0
		0	1	1	0	0
		1	0	1	1	0
		1	1	X	X	X
1	0	0	0	1	0	0
		0	1	1	1	0
		1	0	1	1	0
		1	1	X	X	X
1	1	0	0	1	1	1
		0	1	1	1	1
		1	0	1	1	1
		1	1	X	X	X

SIMPLIFICATION USING K-MAPS

$DN \backslash Q_1 Q_0$		Q_1			
		00	01	11	10
00	0	0	1	1	
01	0	1	1	1	
11	X	X	X	X	
10	1	1	1	1	

N (rows 00, 01)
 D (rows 11, 10)
 Q_0 (columns 01, 11)

K-map for D_1

$$D_1 = Q_1 + D + Q_0 \cdot N$$

$DN \backslash Q_1 Q_0$		Q_1			
		00	01	11	10
00	0	1	1	0	
01	1	0	1	1	
11	X	X	X	X	
10	0	1	1	1	

N (rows 00, 01)
 D (rows 11, 10)
 Q_0 (columns 01, 11)

K-map for D_0

$$D_0 = N \cdot \bar{Q}_0 + Q_0 \cdot \bar{N} + Q_1 \cdot N + Q_1 \cdot D$$

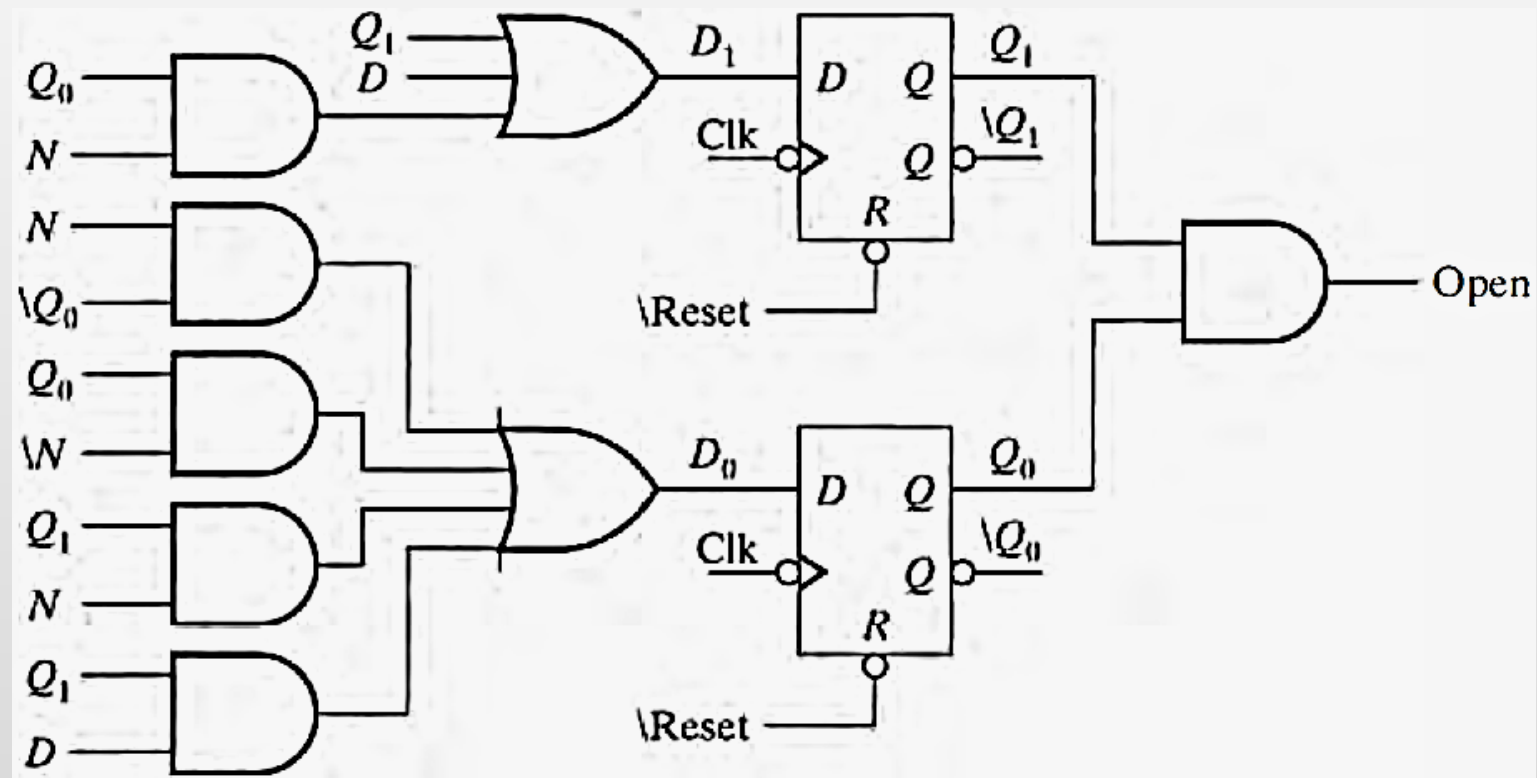
$DN \backslash Q_1 Q_0$		Q_1			
		00	01	11	10
00	0	0	1	0	
01	0	0	1	0	
11	X	X	X	X	
10	0	0	1	0	

N (rows 00, 01)
 D (rows 11, 10)
 Q_0 (columns 01, 11)

K-map for Open

$$\text{Open} = Q_1 \cdot Q_0$$

IMPLEMENTATION



MOORE VS MEALY MACHINE

